



NPTEL Online Certification Course
Applied Accelerated Artificial Architecture
Assignment 2 : Detailed Solution
Indian Institute of Technology Guwahati

Question 1:

Which of the following best describes the key structural difference between a Docker Image and a Container?

Options:

- A. An image consists of a mutable filesystem, whereas a container is an immutable snapshot that cannot be modified.
- B. An image contains multiple layers and metadata, while a container is an instance of that image which adds its own writable layer, namespaces, and cgroups.
- C. A container is built exactly once using a Dockerfile step, whereas an image can be dynamically scaled using cgroups.
- D. Images use shared filesystem diffs to run, while containers completely copy the entire 5–10 GB base filesystem to run independently.

Correct Answer: B

Explanation:

A Docker image is a read-only, immutable template made up of stacked filesystem layers and config metadata. A container is a runnable instance of that image. When a container starts, the Docker engine adds a thin, temporary writable layer (container layer) on top of the image stacks and isolates the process using Linux namespaces (for environment isolation) and cgroups (for resource limiting).

Question 2 :

Why are layer caching and the ordering of steps considered critical when building large AI images?

Options:

- A. Layer caching prevents the container's writable layer from exceeding the 5–10 GB limit.
- B. Incorrect step ordering deletes the underlying image metadata, forcing the runtime to crash.
- C. They drastically reduce rebuild times (e.g., from 30 minutes to 30 seconds) by reusing unchanged filesystem diffs shared between images.

D. They turn a stopped container instance back into an immutable image.

Correct Answer: C

Explanation:

Docker builds images layer-by-layer. If a layer's contents and instructions have not changed, Docker reuses the cached layer. However, if a step changes (like modifying local application code), that cache and all subsequent layer caches are invalidated. Putting slow-changing, heavy steps (like installing CUDA packages or python dependencies) *before* fast-changing steps (like copying application source code) ensures that heavy installations are cached, dramatically speeding up subsequent builds.

Question 3:

The driver version must be old enough for the toolkit in the image.

Options:

- A. TRUE
- B. FALSE

Correct Answer: B

Explanation:

This statement is false. The NVIDIA host driver (kernel-space driver installed on the host operating system) must be new enough to support the CUDA driver API version required by the CUDA toolkit/libraries packaged inside the container image. Host drivers are generally backward-compatible, meaning newer host drivers can run containers with older CUDA toolkits, but old host drivers cannot run newer CUDA toolkit versions.

Question 4 :

Does Docker give us control to run any program on any specific available GPU?

Options:

- A. TRUE
- B. FALSE

Correct Answer: A

Explanation:

Docker, in tandem with the NVIDIA Container Toolkit, gives you granular control over hardware access. You can isolate and target specific GPUs for a container using flags such as `--gpus "device=0"` or by passing the `NVIDIA_VISIBLE_DEVICES` environment variable, restricting the containerized application's visibility to only the selected physical GPUs.

Question 5 :

What is the correct command to install the NVIDIA Container Toolkit?

Options:

- A. `sudo apt-get install -y nvidia-container-toolkit`
- B. `sudo install -y nvidia-container-toolkit`
- C. `sudo apt-get install -y nvidia-containertoolkit`
- D. `sudo apt-get install -a nvidia-container-toolkits`

Correct Answer: A

Explanation:

The correct package name in the official NVIDIA apt repositories is `nvidia-container-toolkit`. Using the standard Debian/Ubuntu package manager `apt-get` with the `-y` (assume-yes) flag is the standard and correct syntax to install the utility.

Question 6 :

A Docker reproducibility bug occurs when local code hasn't changed, but "something changed in the resolver's world." Which scenario best exemplifies this?

Options:

- A. A non-deterministic race condition triggers in the application code.
- B. A corrupted local Docker cache forces the engine to skip build layers.
- C. An unpinned dependency or base image tag pulls a newer, breaking version during a fresh build.
- D. A host OS kernel update changes how the container handles low-level resource limits.

Correct Answer: C

Explanation:

If a Dockerfile points to a dynamic tag (like `ubuntu:latest` or `python:3.10`) or installs packages without strict version locking (like `pip install requirements.txt` with unpinned packages), a rebuild at a later date will query external registries. If an upstream maintainer updates that tag or package with breaking changes, your build will fail or break despite your local application code remaining completely identical.

Question 7 :

Even after pinning human-readable tags, Docker builds can still suffer from reproducibility bugs. Which technique guarantees byte-level build reproducibility?

Options:

- A. Using semantic version ranges (e.g., `>=3.1.5`) in the Dockerfile.

- B. Enabling strict tag-locking in the host daemon settings.
- C. Appending a `.lock` extension to the target build image name.
- D. Referencing the base image by its immutable SHA-256 digest.

Correct Answer: D

Explanation:

Human-readable tags (like `python:3.9-slim`) are mutable—they can be updated by registry owners to include patches or changes. To guarantee absolute, byte-for-byte reproducibility, you must reference the unique, cryptographic SHA-256 hash of the image manifest (e.g., `python:3.9-slim@sha256:...`). Because cryptographic digests are immutable, the engine will always pull the exact same set of image bytes.

Question 8 :

When using `COPY --from=builder`, what happens to the layers of the `builder` stage in the final image?

Options:

- A. They are stored as metadata but don't take up disk space.
- B. They are compressed and hidden within the final layer.
- C. They are completely excluded from the final image's manifest.
- D. They become read-only parent layers for the production stage.

Correct Answer: C

Explanation:

In a multi-stage build, each `FROM` instruction begins a brand new stage with a clean filesystem. When you use `COPY --from=builder`, you are selectively picking out only the specified artifacts/files from that prior stage. The intermediate layers created during the `builder` stage (such as compiler dependencies, build tools, and temporary caches) are discarded entirely and are not referenced in the final image's manifest, resulting in a lean production image.

Question 9 :

What is the correct command to generate a reproducible `requirements.txt` file with dependency hashes using the `pip-tools` suite?

Options:

- A. `pip-compile --generate-hashes --output-file=requirements.txt requirements.in`
- B. `pip-compile --lock-dependencies --export=requirements.txt requirements.in`
- C. `pip-compile --freeze-hashes --output requirements.txt requirements.in`
- D. `pip-compile --strict-verification --write-to requirements.txt requirements.in`

Correct Answer: A

Explanation:

The official CLI tool `pip-compile` uses the `--generate-hashes` flag to parse input requirements (such as `requirements.in`), resolve dependencies, and download and write the security SHA-256 hashes for each resolved package. The `--output-file=` option dictates where the fully locked dependencies file should be written.

Question 10 :

What is the Docker mechanism called when you link a specific, absolute path on your host hard disk directly to a path inside a container to persist your files even after the container is stopped?

Options:

- A. Docker Cache Layers
- B. Host Port Mapping
- C. Container Image Commits
- D. Bind Mounts

Correct Answer: D

Explanation:

A Bind Mount maps a user-specified, absolute path from the host computer directly to a mount point inside the container. Because it references the host filesystem, any modifications made within the container write directly to your local hard drive. This keeps your physical files intact, editable, and saved even if the container is stopped, deleted, or rebuilt.

Question 11 :

When using Unified Memory in CUDA, how does `cudaMallocManaged` handle memory management between the CPU and the GPU?

Options:

- A. It allocates a single pointer accessible by both the CPU and GPU, automatically migrating data between them as needed.
- B. It creates two completely different pointers that require manual synchronization functions after every operation.
- C. It locks the pointer so only the CPU can read the data while the GPU is actively executing a kernel.
- D. It duplicates the pointer across the physical hard drives of both the host system and the graphics card.

Correct Answer: A

Explanation:

`cudaMallocManaged` allocates Unified Memory, which returns a single pointer that can be referenced by both CPU and GPU code. The CUDA driver and system software automatically handle page faults and migrate data between the system RAM (CPU) and video memory (GPU) on demand, removing the need for developers to write manual data transfer commands like `cudaMemcpy`.

Question 12 :

When launching a CUDA kernel using the execution configuration syntax `kernel<<X, Y>>`, what do the parameters **X** and **Y** dictate to the GPU hardware?

Options:

- A. **X** is the number of thread blocks in the grid, and **Y** is the number of parallel threads within each block.
- B. **X** is the number of threads per block, and **Y** is the number of thread blocks in the grid.
- C. **X** is the amount of allocated constant memory, and **Y** is the amount of allocated shared memory.
- D. **X** is the maximum loop count per thread, and **Y** is the number of physical GPU streaming multiprocessors (SMs) to engage.

Correct Answer: A**Explanation:**

In CUDA's triple angle bracket launch configuration `<<gridDim, blockDim>>`:

- The first parameter (**X**) represents the dimension of the grid, which specifies the total number of thread blocks to spawn.
- The second parameter (**Y**) represents the dimension of a block, which specifies the number of threads inside each individual thread block.

Question 13 :

Which GPU sharing mechanism provides strict hardware-level isolation, ensuring that dedicated memory and SMs are completely partitioned so one tenant cannot trigger an Out-of-Memory (OOM) error in another?

Options:

- A. Time-slicing
- B. Multi-Process Service (MPS)
- C. Multi-Instance GPU (MIG)
- D. Dynamic Core Over-provisioning

Correct Answer: C**Explanation:**

Multi-Instance GPU (MIG) splits a single physical GPU into several separate, independent hardware instances (up to 7 on architectures like the H100 or B200). Each partition has dedicated, isolated memory, memory controllers, and Streaming Multiprocessors (SMs). This strict hardware separation ensures that high resource usage or an OOM crash in one instance has absolutely no effect on neighboring instances.

Question 14 :

What is a major isolation limitation when using CUDA Multi-Process Service (MPS) to share a GPU between different processes?

Options:

- A. It completely stops multiple processes from submitting kernels at the same time.
- B. Memory is still shared, meaning a buggy process can corrupt another's memory or crash the entire MPS server.
- C. It drops small inference request throughput by 30-50%.
- D. It enforces strict hardware-level partitions that prevent dynamic memory allocation entirely.

Correct Answer: B

Explanation:

Unlike MIG, MPS uses logical partitioning (soft isolation). Multiple processes feed commands to a single control server to share SM hardware concurrently. However, the physical memory space is not strictly partitioned in hardware. Therefore, a memory access bug or an OOM crash in one client process can corrupt the address space of other active processes or crash the central MPS server daemon, taking all sibling client processes down with it.

Question 15 :

How does gang scheduling solve the "stranded GPU" problem in multi-node clusters?

Options:

- A. It kills running idle nodes if they take too long to start.
- B. It splits a large 4-node job into smaller, single-node jobs.
- C. It ensures all required nodes launch simultaneously, allowing no partial allocations.
- D. It automatically downloads missing training data to the cluster.

Correct Answer: C

Explanation:

In distributed training across multiple nodes, a job cannot run unless all nodes are online and synchronized. The "stranded GPU" issue occurs when some nodes are allocated and sit idle, locking up precious resources while waiting on a missing node that is still queued. Gang scheduling (as used in schedulers like Slurm) avoids this by making an "all-or-nothing" decision: it only runs the job when every single required node is simultaneously ready, eliminating wasteful partial allocations.

Indian Institute of Technology Guwahati